

CORE 07 — Software Engineering

Shared-core file 7 of the NPSC CTSE 2026 prep set.

Three depths for every topic: **one-line fact** (MCQ), **5-mark answer** (~5–6 points),

10/15-mark answer (worked explanation + diagram).

0. Why this matters — where software engineering appears in your three papers

Elective	Where	How heavy
CS Degree	P-II §2 — "Concept of systems, Software development process modules, software project planning and management; cost estimation methods , scheduling, requirement analysis, software testing strategies , Quality Assurance ." Names cost estimation, scheduling and QA explicitly.	Heavy
CS Diploma	P-II §4 (System Analysis & Design) — "System concept: definition, characteristics, elements; Types: Physical, Abstract, Open, Closed; Information system; SDLC: different phases/stages , Role of the System Analyst, Factors affecting failure in SDLC."	Moderate–heavy
Computer Forensic	TP-II §3 — "SDLC: Steps, Waterfall model , Prototypes , Spiral model ; Software Metrics ; Software Project Management; Software Design (system, detailed, function-oriented, object-oriented, UI, design-level metrics); Coding and Testing (testing-level metrics); Software quality and reliability , clean-room approach, software reengineering , reverse engineering ." The most detailed of the three.	Heavy

This is a strong shared topic and a "safe" one — it is descriptive, diagram-friendly and rewards structured answers, so it yields Section-A marks in every paper. The two numerical islands — **COCOMO / function points** and **PERT-CPM / cyclomatic complexity** — are the highest-return practice, because worked numericals are unambiguous to mark. Study the process models, testing and cost estimation deeply; they recur across all three electives.

Topic checklist

- ☐ System concept, characteristics, elements; types (physical/abstract, open/closed)
- ☐ Software, its characteristics; the **software crisis**
- ☐ **SDLC phases** in order
- ☐ **Process models**: Waterfall, Prototype, Spiral, Incremental, Iterative, RAD, V-model, Agile — each with a diagram, when-to-use, pros and cons
- ☐ **SRS** — contents, characteristics of a good SRS
- ☐ Feasibility study (TELOS)
- ☐ Project planning; role of the system analyst; failure factors
- ☐ **Cost estimation — COCOMO basic and intermediate (worked); function points (worked)**
- ☐ Scheduling: **Gantt charts**, **PERT/CPM with a worked critical path**
- ☐ Design: function- vs object-oriented, modularity, abstraction
- ☐ **Coupling and cohesion — every type ranked, with examples**
- ☐ User-interface design principles
- ☐ **Testing**: unit, integration (top-down/bottom-up/sandwich), system, acceptance
- ☐ **Black-box**: equivalence partitioning, BVA, cause-effect, decision tables
- ☐ **White-box**: statement/branch/path coverage, **cyclomatic complexity (worked)**
- ☐ Alpha/beta/regression testing; **V&V**
- ☐ Software metrics (LOC, FP, Halstead)

- [] SQA; quality models — **McCall, ISO 9126, CMM levels 1–5**
- [] Software reliability (MTBF, MTTF, MTTR)
- [] Maintenance types; reengineering and reverse engineering

1. Systems, software and the software crisis

Concept — what a system is (Diploma §4)

A **system** is an **organised set of interrelated components that work together toward a common goal**, transforming **inputs** into **outputs**. Every system has:

- **Inputs** (data/resources), **processing** (the transformation), **outputs** (results).
- **Boundary** (what is inside vs outside), **environment** (what surrounds it), **interfaces** (connections between components/other systems).
- **Feedback and control** (comparing output with a goal and adjusting).

Characteristics of a system: organisation, interaction, interdependence, integration, and a central objective.

Types of system (a Diploma 5-marker):

Pair	Distinction
Physical vs abstract	Physical = tangible (computers, people); abstract = conceptual (a formula, a model)
Open vs closed	Open = exchanges with its environment (adapts); closed = isolated, self-contained (no environment interaction)
Deterministic vs probabilistic	Deterministic = predictable behaviour (a program); probabilistic = uncertain (weather, inventory demand)
Manual vs automated	Done by people vs by machines

An **information system** is a system that collects, processes, stores and distributes **information** to support decision-making and control (e.g. TPS, MIS, DSS, ESS).

Software and its characteristics

Software is not just code: it is **programs + data + documentation** — the set of instructions and associated artefacts that make a computer do useful work. Its **characteristics** distinguish it from hardware:

- Software is **developed/engineered, not manufactured** — cost is in design, not production.
- Software **does not wear out** — but it **deteriorates** through repeated changes (the "bathtub curve" for hardware rises again at end-of-life; software's failure rate would be flat except that each change introduces new defects — an "**increasing failure rate due to changes**").
- Most software is **custom-built**, not assembled from standard components (though reuse is rising).
- Software is **intangible and complex**.

Types: system software (OS, compilers), application software, engineering/scientific, embedded, product-line, web, AI software.

The software crisis

The **software crisis** (term coined at the 1968 NATO conference) is the historical realisation that software projects routinely came in **over budget, over time, of poor quality, hard to maintain, and often failed outright**, because ad-hoc "code-and-fix" development could not scale

to large systems. Its symptoms are the standard 5-mark list:

- Projects **exceeded budget and schedule**.
- Software was of **poor quality / unreliable**.
- Software was **difficult to maintain and did not meet requirements**.
- Productivity could not keep up with demand.

Software engineering is the disciplined response: *"the application of a systematic, disciplined, quantifiable approach to the development, operation and maintenance of software"* (IEEE). It brings engineering principles — process, measurement, quality control — to software.

Likely exam questions

- **[5]** Define a system. State its characteristics and elements.
- **[5]** Differentiate between open and closed / physical and abstract systems.
- **[5]** What is the software crisis? State its causes and symptoms.
- **[5]** What are the characteristics of software? How does it differ from hardware?
- **[10]** What is software engineering? Explain the software crisis that led to it and the characteristics of software.

MCQ traps

Trap	Correct answer
"Software wears out?"	No — it deteriorates (due to changes), it does not wear out.
"Software = ?"	Programs + data + documentation , not just code.
"A system that interacts with its environment is?"	Open .
"The term 'software crisis' arose at?"	The 1968 NATO conference .
"Software is manufactured?"	No — engineered/developed .

2. The Software Development Life Cycle (SDLC)

Concept

The **SDLC** is the structured sequence of **phases** a software project passes through from conception to retirement. The *process models* in §3 are different **arrangements** of these same phases; learn the phases once and every model becomes a rearrangement of them.

#	Phase	What happens	Output artefact
1	Requirement gathering & analysis	Elicit, analyse and document what the system must do	SRS document
2	Feasibility study (often merged with 1)	Check technical/economic/legal viability	Feasibility report
3	Design	Architecture, modules, data, interfaces (HLD then LLD)	Design document (SDD)
4	Coding / implementation	Translate design into source code	Source code
5	Testing	Verify and validate — find and fix defects	Test reports, working software

6	Deployment / installation	Release to the user environment	Deployed system
7	Maintenance	Correct, adapt, enhance after delivery	Updated system

The maintenance phase is the longest and most expensive — commonly **60–70% of total lifetime cost**. That single fact is worth a mark and reframes the whole subject: the point of good engineering is to reduce maintenance cost.

Role of the system analyst (Diploma): the bridge between users and developers — studies the existing system, elicits and analyses requirements, prepares the SRS, designs the logical solution, and guides implementation. Skills: analytical, communication, technical, and domain knowledge.

Factors causing SDLC failure (Diploma): unclear/changing requirements, poor planning and estimation, inadequate user involvement, weak project management, unrealistic schedules, insufficient testing, and poor communication.

Likely exam questions

- **[5]** What is the SDLC? List its phases in order.
- **[5]** Explain the role of the system analyst.
- **[5]** State the factors that cause failure in the SDLC.
- **[10]** Explain the phases of the SDLC with the activities and output of each phase.

MCQ traps

Trap	Correct answer
"First phase of the SDLC?"	Requirement gathering & analysis.
"Most expensive / longest phase?"	Maintenance.
"Output of the requirement phase?"	The SRS .
"HLD and LLD belong to which phase?"	Design.

3. Software process models — compared

Concept

A **process (life-cycle) model** prescribes the **order and manner** in which SDLC phases are carried out. There is no universally best model; the choice depends on **requirement clarity, project size, risk, and how much the customer needs to be involved**. This comparison is the single most examined SE topic — expect a 10- or 15-marker asking you to explain several models with diagrams, pros/cons and when to use each.

3.1 Waterfall model

Idea: phases flow **strictly downward**, each completed and signed off before the next begins — like water cascading. No going back (in the pure form).

Diagram to draw: a staircase of boxes descending left-to-right — Requirements → Design → Implementation → Testing → Deployment → Maintenance — each feeding the next.

```

Requirements
  └─> Design
      └─> Implementation
          └─> Testing

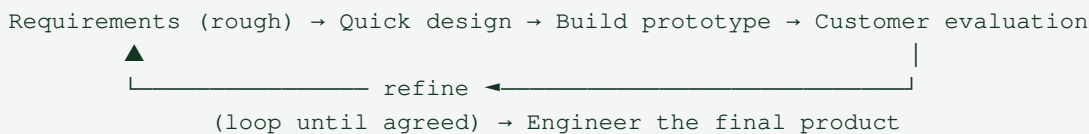
```



When to use	Pros	Cons
Requirements are clear, fixed and well understood ; short, small projects; mature domains	Simple, easy to manage ; clear milestones and documentation; disciplined	No working software until late ; very poor at handling changing requirements ; risk found late ; customer sees the product only at the end

3.2 Prototype model

Idea: build a **quick, throwaway (or evolutionary) working model** early, show it to the customer, gather feedback, and **refine iteratively** until requirements are clear — *then* build the real system. Best when **requirements are unclear or the customer cannot articulate them**.



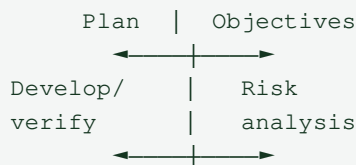
When to use	Pros	Cons
Requirements unclear/evolving ; user needs early visualisation; new/experimental domains	Early user feedback ; reduces requirement risk; users see something concrete quickly	Throwaway effort; customer may mistake the prototype for the product; poor documentation ; can invite scope creep

3.3 Spiral model (Boehm, 1986)

Idea: a **risk-driven** model that combines iterative prototyping with the systematic waterfall. The project spirals outward through repeated cycles, each with **four quadrants**:

1. **Determine objectives**, alternatives, constraints.
2. **Identify and resolve risks** (prototyping, analysis) — *the defining feature*.
3. **Develop and verify** the next-level product.
4. **Plan** the next iteration.

Each loop produces a more complete version; **the radius represents cost, the angle represents progress**.



When to use	Pros	Cons
Large, expensive, high-risk projects ; requirements evolve; R&D	Explicit risk management ; flexible; supports change; early prototypes	Costly and complex ; needs risk-assessment expertise ; overkill for small projects; hard to estimate

3.4 Incremental model

Idea: build the system in **increments** — deliver a working core first, then add functionality in successive, deliverable pieces. Each increment goes through its own design-code-test mini-cycle. **The customer gets usable software early** and each increment adds capability.

When to use	Pros	Cons
Core requirements clear, extras can follow; need early delivery	Early working product ; easier to test/debug per increment; lower initial delivery risk	Needs good architecture up front; total cost may be higher; integration overhead

3.5 Iterative (enhancement) model

Idea: start with a **simple implementation of a subset** of requirements and **iteratively enhance** it, **refining the same product** version after version until complete. (Distinction from incremental: incremental **adds new pieces**; iterative **refines the whole** repeatedly. In practice most modern methods are **iterative and incremental**.)

When to use	Pros	Cons
Requirements understood but large; want early feedback	Feedback each cycle; changes accommodated; early risk visibility	Re-work each iteration; needs more resources/management

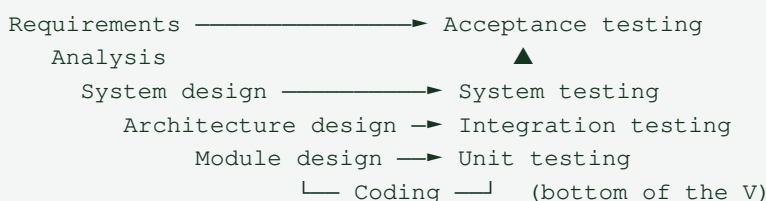
3.6 RAD — Rapid Application Development

Idea: compress the schedule by using **component reuse, powerful CASE/4GL tools, and parallel development by multiple teams**, each building a component in a short **time-box** (60–90 days), then integrating. Heavy user involvement via workshops (JAD).

When to use	Pros	Cons
Requirements reasonably known; tight deadline ; reusable components exist; modular system	Very fast delivery ; reuse; strong user involvement	Needs skilled developers and strong tooling ; needs a modular, componentisable system; not for large, complex or high-performance systems

3.7 V-model (Verification and Validation model)

Idea: an extension of waterfall where **each development phase has a corresponding testing phase**, drawn as a **V**. The **left arm** goes down through the development phases; the **right arm** rises through the matching test phases; the two arms are linked so that test plans are written **during** the corresponding development phase — **testing is planned in parallel with development**, not deferred.



When to use	Pros	Cons
Requirements clear and fixed; small/medium projects where quality is critical (medical, defence)	Testing planned early ; each phase verified; disciplined, high quality	Rigid like waterfall; poor at change ; no early prototype

3.8 Agile

Idea: deliver working software in short **iterations (sprints, ~2–4 weeks)**, embracing **changing requirements**, continuous customer collaboration, and self-organising teams. Guided by the **Agile Manifesto (2001)**: *individuals and interactions over processes and tools; working software over documentation; customer collaboration over contract negotiation*;

responding to change over following a plan. Frameworks: **Scrum** (sprints, product/sprint backlog, daily stand-up, roles: Product Owner, Scrum Master, Team), **XP** (pair programming, TDD, continuous integration), **Kanban**.

When to use	Pros	Cons
Requirements volatile ; need frequent delivery; engaged customer; small–medium teams	Adapts to change ; continuous delivery; high customer satisfaction; early ROI	Less documentation ; hard to scale to very large teams; needs experienced, disciplined teams; unpredictable final scope/cost

The master comparison table — reproduce this

Model	Requirements	Risk handling	Customer involvement	Working software	Best for
Waterfall	Fixed, clear	Poor (late)	Low (start & end)	Late	Small, well-understood projects
Prototype	Unclear	Medium	High	Early (prototype)	Vague requirements
Spiral	Evolving	Excellent (risk-driven)	Medium	Each spiral	Large, high-risk projects
Incremental	Core clear	Good	Medium–high	Early, in increments	Need early partial delivery
Iterative	Understood, large	Good	Medium	Each iteration	Large systems, evolving detail
RAD	Known, modular	Medium	High (workshops)	Very fast	Tight deadline, reusable parts
V-model	Fixed, clear	Medium	Low	Late	Quality-critical, stable reqs
Agile	Volatile	Good (continuous)	Very high	Continuous	Changing requirements

Likely exam questions

- **[5]** Explain the Waterfall model with a diagram, stating two advantages and two drawbacks.
- **[5]** When is the prototype model preferred over the waterfall model?
- **[5]** Explain the spiral model. Why is it called risk-driven?
- **[5]** Differentiate between the incremental and iterative models.
- **[5]** Explain the V-model. How does it improve on the waterfall model?
- **[5]** State the four values of the Agile Manifesto.
- **[10]** Compare the Waterfall, Prototype and Spiral models with diagrams, pros, cons and when to use each.
- **[15]** Explain the various software process models (Waterfall, Prototype, Spiral, Incremental, RAD, V-model, Agile) with diagrams, and recommend a suitable model for (a) a well-understood payroll system, (b) a research project with unclear requirements, and (c) a large high-risk banking system, justifying each choice.

MCQ traps

--

Trap	Correct answer
"Which model is 'risk-driven'?"	Spiral (Boehm, 1986).
"Which model has no working software until the end?"	Waterfall .
"Which model pairs each dev phase with a test phase?"	V-model .
"Which model builds a throwaway model first?"	Prototype .
"Which best handles changing requirements?"	Agile .
"RAD relies on?"	Reusable components and CASE/4GL tools .
"Incremental vs iterative — incremental?"	Adds new features ; iterative refines the whole.
"Scrum roles?"	Product Owner, Scrum Master, Team .

4. Software Requirements Specification (SRS)

Concept

The **SRS** is the official document that records **what the system must do** (functional requirements) and **how well** (non-functional requirements). It is the **contract** between customer and developer and the baseline against which the finished system is validated.

Requirement types:

- **Functional requirements** — specific behaviours/services ("the system shall generate a monthly report").
- **Non-functional requirements** — quality attributes and constraints: performance, security, usability, reliability, portability, scalability (the "-ilities").
- **Domain requirements** — arising from the application domain.

Characteristics of a good SRS (a guaranteed 5-marker — the "-ct" list):

Quality	Meaning
Correct	Every requirement truly reflects a need
Complete	All requirements included; nothing missing
Unambiguous	Each requirement has exactly one interpretation
Consistent	No requirement contradicts another
Verifiable / testable	You can devise a test to check it is met
Modifiable	Well-structured so changes are easy
Traceable	Each requirement can be traced to its origin and to design/test
Ranked	Prioritised by importance/stability
Feasible	Achievable within constraints

IEEE 830 is the standard SRS template. The costliest defects are requirement defects — a requirement error found in maintenance can cost **~100×** what it costs to fix during the requirement phase; this is the argument for investing in a good SRS.

Likely exam questions

- [5] What is an SRS? Differentiate functional and non-functional requirements.
- [5] State the characteristics of a good SRS.
- [10] What is requirement analysis? Explain the contents and desirable characteristics of an SRS document.

MCQ traps

Trap	Correct answer
"The system shall respond in 2 seconds' is a?"	Non-functional requirement.
"The SRS is produced in which phase?"	Requirement analysis .
"Standard for SRS?"	IEEE 830 .
"An SRS must be unambiguous means?"	One interpretation only .

5. Feasibility study

Concept

The **feasibility study** decides *whether the project is worth doing* before serious resources are committed. The classic checklist is **TELOS**:

Feasibility	Question
Technical	Can it be built with available technology and skills?
Economic	Do the benefits outweigh the costs? (cost-benefit analysis , ROI, payback)
Legal	Does it comply with laws, licences, contracts, data-protection rules?
Operational	Will it work within the organisation and be accepted by users?
Schedule (time)	Can it be delivered within the required timeframe?

Economic feasibility is usually decisive; it uses cost-benefit analysis, **payback period**, **net present value (NPV)** and **return on investment (ROI)**.

Likely exam questions

- [5] What is a feasibility study? Explain its types (TELOS).
- [5] Explain economic feasibility and cost-benefit analysis.

MCQ traps

Trap	Correct answer
"TELOS stands for?"	Technical, Economic, Legal, Operational, Schedule .
"Cost-benefit analysis is part of?"	Economic feasibility .
"'Will users accept it?' is which feasibility?"	Operational .

6. Project planning and management

Concept

Software project management balances the **project management triangle** — **scope, time, cost** (with quality at the centre). Core planning activities:

- **Scope definition** and **work breakdown structure (WBS)** — decompose the project into tasks.
- **Estimation** — size, effort, cost, time (see §7).
- **Scheduling** — sequence tasks, assign resources (see §8).
- **Risk management** — identify, analyse, prioritise, plan mitigation, monitor.
- **Staffing / resource planning; configuration management** (version control, change control); **monitoring and control**.

Putnam's / Brooks's insight: *"Adding manpower to a late software project makes it later"* (**Brooks's Law**) — because of the communication overhead among $n(n-1)/2$ pairs and ramp-up time. Effort and time are not freely interchangeable.

Likely exam questions

- [5] What is the project management triangle? Explain scope, time and cost.
- [5] State Brooks's Law and explain why it holds.
- [5] What is a work breakdown structure (WBS)?
- [10] Explain the activities involved in software project planning and management.

MCQ traps

Trap	Correct answer
"Adding people to a late project makes it later" is?"	Brooks's Law.
"The project triangle constrains?"	Scope, time, cost.
"Decomposing a project into tasks gives a?"	WBS.

7. Cost estimation — COCOMO and function points

This is a top numerical topic. Work every formula by hand and show every step.

Concept

Estimation predicts the **size**, then the **effort** (person-months), then the **time** and **cost** of a project. Size is measured in **LOC (lines of code)** or **function points (FP)**. The two headline techniques are **COCOMO** (from size) and **Function Point Analysis** (from functionality).

COCOMO — Constructive Cost Model (Boehm, 1981)

COCOMO estimates **effort** and **development time** from the estimated size in **KLOC** (thousands of delivered lines of code). It comes in three project **modes**:

Mode	Nature	Team/size	Example
------	--------	-----------	---------

Organic	Small, simple; experienced team; familiar, stable environment	Small (< ~50 KLOC)	A simple inventory/payroll system
Semi-detached	Medium size and complexity; mixed experience	Medium (~50–300 KLOC)	A DBMS, a compiler, a transaction system
Embedded	Large, complex; tight hardware/software/regulatory constraints	Large (> ~300 KLOC)	Real-time OS, avionics, ATC

Basic COCOMO

Two formulas:

Effort $E = a \times (\text{KLOC})^b$ person-months (PM)

Time $D = c \times (E)^d$ months

Persons required $P = E / D$; productivity = KLOC / E.

Coefficients (**memorise the table** — it is handed out rarely, so learn it):

Mode	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semi-detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

Worked example 1 — Basic COCOMO, organic

A project is estimated at 32 KLOC, organic mode. Find effort, development time, number of people and productivity.

Effort $E = 2.4 \times (32)^{1.05}$

$32^{1.05} = 32 \times 32^{0.05}$. $32^{0.05} = e^{(0.05 \cdot \ln 32)} = e^{(0.05 \times 3.4657)} = e^{0.17329} = 1.1892$.

So $32^{1.05} = 32 \times 1.1892 = 38.05$.

$E = 2.4 \times 38.05 = 91.3 \approx 91$ person-months.

Time $D = 2.5 \times (91.3)^{0.38}$

$91.3^{0.38} = e^{(0.38 \cdot \ln 91.3)} = e^{(0.38 \times 4.514)} = e^{1.7153} = 5.559$.

$D = 2.5 \times 5.559 = 13.9 \approx 14$ months.

People $P = E / D = 91.3 / 13.9 = 6.6 \approx 7$ persons.

Productivity = $32 / 91.3 = 0.35$ KLOC per person-month (≈ 350 LOC/PM).

Worked example 2 — Basic COCOMO, embedded

Size = 400 KLOC, embedded mode.

$E = 3.6 \times (400)^{1.20}$. $400^{1.20} = e^{(1.2 \cdot \ln 400)} = e^{(1.2 \times 5.9915)} = e^{7.1898} = 1329$.

$E = 3.6 \times 1329 = 4784 \approx 4785$ person-months.

$D = 2.5 \times (4784)^{0.32} = 2.5 \times e^{(0.32 \cdot \ln 4784)} = 2.5 \times e^{(0.32 \times 8.473)} = 2.5 \times e^{2.7113} =$

$2.5 \times 15.05 = 37.6 \approx 38$ months. $P = 4784 / 37.6 \approx 127$ persons.

Intermediate COCOMO

Basic COCOMO ignores project-specific factors. **Intermediate COCOMO** multiplies the nominal effort by an **Effort Adjustment Factor (EAF)** — the product of **15 cost drivers** rated from *very low* to *extra high*, grouped as **product, hardware, personnel and project** attributes (e.g. required reliability RELY, database size DATA, complexity CPLX, analyst capability ACAP, programmer capability PCAP, use of tools TOOL, schedule SCED).

$$E = a \times (KLOC)^b \times EAF$$

with intermediate coefficients:

Mode	a	b
Organic	3.2	1.05
Semi-detached	3.0	1.12
Embedded	2.8	1.20

EAF = product of the 15 cost-driver multipliers; each rating maps to a number (nominal = 1.0, higher reliability > 1, better staff < 1). EAF typically ranges ~0.9 to ~1.4.

Worked example 3 — Intermediate COCOMO

Organic, 32 KLOC. Suppose the selected cost drivers give multipliers whose product is EAF = 1.15.

Nominal $E = 3.2 \times (32)^{1.05} = 3.2 \times 38.05 = 121.8$.

Adjusted $E = 121.8 \times 1.15 = 140.0$ person-months.

$D = 2.5 \times (140)^{0.38} = 2.5 \times e^{(0.38 \times 4.942)} = 2.5 \times e^{1.878} = 2.5 \times 6.54 = 16.4 \approx 16$ months.

(Detailed COCOMO goes further, applying cost drivers phase by phase — mention it exists.)

Function Point Analysis (FPA)

Problem COCOMO has: LOC is **language-dependent** and unknown until late. **Function points** measure the **size from the functionality delivered to the user**, independent of language, and can be computed **early from the requirements**.

Five function types, each counted and weighted by complexity (simple/average/complex):

Component	Meaning	Simple	Avg	Complex
EI — External Inputs	User inputs that update data (forms)	3	4	6
EO — External Outputs	Outputs to the user (reports, messages)	4	5	7
EQ — External Inquiries	Input–output queries (no update)	3	4	6
ILF — Internal Logical Files	Internal data groups maintained	7	10	15
EIF — External Interface Files	Data used but maintained elsewhere	5	7	10

Steps:

1. **UFP (Unadjusted Function Points)** = Σ (count $\times$ weight) over all five types.
2. **VAF (Value Adjustment Factor)** = $0.65 + 0.01 \times \Sigma Fi$, where the **14 general system characteristics** are each rated 0–5 (ΣFi ranges 0–70, so VAF ranges **0.65 to 1.35**).
3. **FP = UFP $\times$ VAF.**
4. Optionally, **LOC = FP $\times$ (LOC per FP for the language)** to feed COCOMO.

Worked example 4 — Function points

A system has: EI = 22 (avg), EO = 20 (avg), EQ = 10 (simple), ILF = 4 (avg), EIF = 2 (avg). Total degree of influence $\Sigma Fi = 30$.

Step 1 — UFP (use the weights above):

Type	Count	Weight	Product
------	-------	--------	---------

EI (avg)	22	4	88
EO (avg)	20	5	100
EQ (simple)	10	3	30
ILF (avg)	4	10	40
EIF (avg)	2	7	14
		UFP	272

Step 2 — VAF = $0.65 + 0.01 \times 30 = 0.65 + 0.30 = \mathbf{0.95}$.

Step 3 — FP = $272 \times 0.95 = \mathbf{258.4 \approx 258 \text{ function points}}$.

Step 4 — to LOC (say the language averages 50 LOC/FP):

LOC = $258 \times 50 = 12\,900 \approx \mathbf{12.9 \text{ KLOC}}$, which could then be fed into COCOMO.

Likely exam questions

- **[5]** Explain the three modes of COCOMO with examples.
- **[5]** State the basic COCOMO formulas and their coefficient table.
- **[5]** Differentiate basic and intermediate COCOMO. What is the EAF?
- **[5]** What are function points? List the five function types.
- **[10]** For an organic project of 40 KLOC, compute the effort, development time and number of people using basic COCOMO. ($E = 2.4 \times 40^{1.05} = 2.4 \times 46.9 = 112.6 \text{ PM}$; $D = 2.5 \times 112.6^{0.38} = 2.5 \times 5.98 = 15.0 \text{ months}$; $P \approx 8$.)
- **[10]** Compute the function-point count for the given system and convert it to LOC.
- **[15]** Explain software cost estimation. Describe COCOMO (basic and intermediate) and Function Point Analysis, and solve one numerical of each.

MCQ traps

Trap	Correct answer
"COCOMO estimates from?"	KLOC (size) .
"Organic basic coefficients a, b?"	2.4 and 1.05 .
"Which mode has the highest exponent b?"	Embedded (1.20) .
"Intermediate COCOMO adds the?"	EAF (from 15 cost drivers) .
"Function points depend on the programming language?"	No — they are language-independent.
"How many function-point types?"	Five (EI, EO, EQ, ILF, EIF).
"VAF range?"	0.65 to 1.35 .
"Who developed COCOMO?"	Barry Boehm (1981) .

8. Project scheduling — Gantt, PERT and CPM

Concept

Scheduling sequences the estimated tasks over calendar time and assigns resources. Two visual tools dominate the exam: the **Gantt chart** (simple bar timeline) and the **network diagram** analysed with **PERT/CPM** (dependencies and the critical path).

Gantt chart

A **horizontal bar chart**: each task is a bar whose **length = duration** and **position = start and end dates**; bars can overlap to show parallel work, and **milestones** are marked as diamonds. It shows **what runs when** and **progress** at a glance, but **does not clearly show task dependencies** — that is what PERT/CPM adds.

PERT and CPM

Both use an **activity network** (nodes and arrows) to find the **critical path** — the longest path through the project, which **determines the minimum project duration**. Any delay on a critical activity delays the whole project.

	CPM (Critical Path Method)	PERT (Program Evaluation & Review Technique)
Time estimate	Single, deterministic duration	Three estimates → probabilistic
Focus	Time–cost trade-off (crashing)	Uncertainty / risk in scheduling
Suited to	Repetitive, well-known projects (construction)	Research/new projects with uncertain times
Activities	Activity-on-node commonly	Activity-on-arrow commonly

PERT expected time for an activity from optimistic (o), most likely (m), pessimistic (p):

$$t_e = (o + 4m + p) / 6, \text{ variance } \sigma^2 = ((p - o)/6)^2$$

Scheduling terms: **ES** (earliest start), **EF** (earliest finish = ES + duration), **LS** (latest start), **LF** (latest finish). **Slack/float** = **LS – ES** = **LF – EF**. **Critical activities have zero slack**. A **forward pass** computes ES/EF; a **backward pass** computes LS/LF.

Worked example — critical path

Activities, durations (weeks) and predecessors:

Activity	Duration	Predecessor
A	3	—
B	4	A
C	2	A
D	5	B
E	3	C
F	2	D, E

Forward pass (ES, EF):

- A: ES 0, EF 3.
- B: ES 3, EF 7. C: ES 3, EF 5.
- D: ES 7, EF 12. E: ES 5, EF 8.

- F: ES = max(EF of D, E) = max(12, 8) = 12, EF = 14.

Project duration = 14 weeks.

Paths and lengths:

- A → B → D → F = 3+4+5+2 = **14** ← longest

- A → C → E → F = 3+2+3+2 = 10

Critical path = A → B → D → F (14 weeks). Activities C and E have slack: E's slack = 12 - 8 = **4 weeks**, so E can slip up to 4 weeks without delaying the project. C and E are **non-critical**.

If asked to "crash": to shorten the project you must shorten a **critical** activity (A, B, D or F); crashing C or E achieves nothing until the critical path changes.

Likely exam questions

- **[5]** Differentiate between a Gantt chart and a PERT chart.
- **[5]** Differentiate between PERT and CPM.
- **[5]** Define critical path, slack/float, ES, EF, LS, LF.
- **[5]** State the PERT expected-time formula and compute t_e for $o=2$, $m=4$, $p=12$. ($t_e = (2+16+12)/6 = 5$.)
- **[10]** For the given activity table, draw the network, compute ES/EF/LS/LF, find the critical path and the project duration, and state the slack of each activity.

MCQ traps

Trap	Correct answer
"The critical path is the?"	Longest path; it sets the minimum project duration.
"Slack on a critical activity?"	Zero .
"PERT uses how many time estimates?"	Three (o, m, p).
"PERT expected time formula?"	$(o + 4m + p)/6$.
"Which chart shows dependencies best?"	PERT/CPM network (not Gantt).
"CPM emphasises?"	Time–cost trade-off .

9. Software design

Concept

Design transforms the *what* of the SRS into the *how* — the blueprint for coding. It has two levels: **High-Level Design (HLD / architectural)** — modules and their relationships — and **Low-Level Design (LLD / detailed)** — the internal logic of each module. Good design rests on **abstraction, modularity, information hiding** and low coupling / high cohesion.

Fundamental design concepts:

Concept	Meaning
Abstraction	Focus on essential features, hide detail (procedural, data, control abstraction)
Modularity	Divide the system into separately named, addressable modules

Information hiding	Each module hides its internal data/algorithm behind an interface
Stepwise refinement	Top-down elaboration of detail
Refactoring	Improving internal structure without changing external behaviour

Modularity is desirable because it makes a system easier to understand, test and maintain — but *too many* tiny modules raise integration cost, and *too few* large ones are hard to understand, so there is an **optimum number of modules** minimising total cost.

Function-oriented vs object-oriented design

	Function-oriented (procedural)	Object-oriented
Decomposition	By functions/processes (top-down)	By objects = data + behaviour
Central abstraction	The function ; data is separate and shared	The object/class ; data and functions bound together (encapsulation)
Data	Often global/shared, passed around	Encapsulated , hidden inside objects
Tools	DFD , structure charts, data dictionary	Class/object diagrams, UML
Key ideas	Modules, coupling/cohesion	Encapsulation, inheritance, polymorphism, abstraction
Change impact	A data-structure change can ripple widely	Localised behind object interfaces
Suits	Well-defined procedural problems	Large, evolving systems; reuse

The **DFD (Data Flow Diagram)** is the core function-oriented tool: circles = processes, arrows = data flows, open rectangles = data stores, squares = external entities; **Level 0 = context diagram**, refined into Levels 1, 2, ...

Coupling and cohesion — ranked, with examples

The **two central quality measures of a modular design**, and a guaranteed exam question. The design goal is **low coupling and high cohesion**.

Coupling = the **degree of interdependence between modules**. **Low (loose) coupling is better** — modules can be changed, tested and reused independently. Ranked from **best (lowest)** to **worst (highest)**:

Rank	Coupling type	Meaning	Example
1 (best)	Data coupling	Modules share data only through simple parameters (atomic values)	<code>computeTax(income)</code> returns a value
2	Stamp coupling	A whole data structure is passed but only part is used	Passing an entire <code>Employee</code> record to get the salary
3	Control coupling	One module passes a control flag that dictates the other's logic	<code>printReport(flag)</code> where flag chooses the format
4	External coupling	Modules share an externally imposed format/protocol/device	Both tied to a specific file format or hardware

5	Common coupling	Modules share global data	Several modules read/write the same global variable
6 (worst)	Content coupling	One module directly accesses or modifies another's internals	Module A jumps into or changes Module B's local data

Cohesion = the **degree to which the elements *within* a single module belong together**.

High cohesion is better — a module should do **one well-defined thing**. Ranked from **worst (lowest)** to **best (highest)**:

Rank	Cohesion type	Meaning	Example
1 (worst)	Coincidental	Elements grouped arbitrarily , no relationship	A "utilities" module of unrelated functions
2	Logical	Elements do similar kinds of things, chosen by a flag	One module doing "all input", selected by a parameter
3	Temporal	Elements grouped because they run at the same time	An <code>init()</code> that opens files, zeroes counters, sets flags
4	Procedural	Elements follow a sequence of control	Steps executed in order but on unrelated data
5	Communicational	Elements operate on the same data	Functions all working on the same input record
6	Sequential	Output of one element is the input of the next	Read → validate → format the same data in a chain
7 (best)	Functional	All elements contribute to one single, well-defined task	<code>computeSquareRoot(x)</code>

The one-line goal to state: *strive for high (functional) cohesion and low (data)*

coupling. Memory aids for the orders: coupling best→worst = **Data, Stamp, Control, External, Common, Content**; cohesion worst→best = **Coincidental, Logical, Temporal, Procedural, Communicational, Sequential, Functional**.

User-interface (UI) design

Golden rules (Shneiderman / Mandel):

1. **Place the user in control** — flexible, forgiving, interruptible.
2. **Reduce the user's memory load** — recognition over recall, meaningful defaults.
3. **Make the interface consistent** — uniform look, layout and behaviour.

Add principles: **feedback** for every action, **error prevention and easy recovery** (undo), **simplicity/aesthetics**, and **accessibility**. The UI design process is **iterative**: analyse users/tasks → design → prototype → **evaluate with users** → refine.

Likely exam questions

- **[5]** Differentiate between coupling and cohesion. State the design goal.
- **[5]** List the types of coupling from best to worst with an example of each.
- **[5]** List the types of cohesion from worst to best with an example of each.
- **[5]** Differentiate function-oriented and object-oriented design.

- **[5]** State the golden rules of user-interface design.
- **[10]** Explain coupling and cohesion in detail, ranking every type with examples, and state why low coupling and high cohesion are desirable.
- **[10]** Explain the fundamental concepts of software design: abstraction, modularity and information hiding, and the DFD as a design tool.

MCQ traps

Trap	Correct answer
"Design goal for coupling and cohesion?"	Low coupling, high cohesion.
"Best (lowest) coupling?"	Data coupling. Worst = content coupling.
"Best (highest) cohesion?"	Functional cohesion. Worst = coincidental.
"Modules sharing global data have?"	Common coupling.
"'Output of one is input of the next' is which cohesion?"	Sequential.
"Passing a whole record to use one field is?"	Stamp coupling.
"Binding data and functions together is?"	Encapsulation (OO).
"A DFD process is drawn as a?"	Circle/bubble.

10. Software testing

Concept

Testing is the process of **executing a program with the intent of finding errors** (Myers). It can only show the **presence** of defects, never their absence (Dijkstra). Vocabulary to keep straight: an **error** (human mistake) causes a **fault/defect/bug** (in the code) which, when executed, causes a **failure** (wrong behaviour).

Levels of testing

Level	What is tested	By whom	Notes
Unit testing	A single module/function in isolation	Developer	Needs stubs (dummy called modules) and drivers (dummy calling modules)
Integration testing	Interactions between combined modules	Developers/testers	Strategies below
System testing	The complete, integrated system against requirements	Independent testers	Functional + non-functional (performance, security, load, stress, usability)
Acceptance testing	The system against user needs, for sign-off	Customer/user	Alpha and beta (below)

Integration strategies — a favourite comparison:

Strategy	How	Needs	Pros / cons
----------	-----	-------	-------------

Top-down	Integrate from the top module downward , replacing lower modules with stubs	Stubs	Major control logic tested early; but lower-level modules tested late, many stubs needed
Bottom-up	Integrate from the lowest modules upward , using drivers	Drivers	Low-level modules tested thoroughly early; but the whole system emerges only at the end, no early prototype
Sandwich (hybrid)	Combine top-down and bottom-up around a target layer	Both stubs and drivers	Balances the two; complex to plan
Big-bang	Integrate everything at once and test	—	Simple but fault localisation is very hard — avoid for large systems

Black-box testing (functional)

Tests **behaviour against the specification**, with **no knowledge of internal code** — inputs in, outputs checked. Techniques:

Technique	Idea	Example
Equivalence Partitioning (EP)	Divide inputs into classes that should behave the same ; test one value per class (one valid, plus invalid classes)	Age 18–60 valid → test one value in 18–60, one < 18, one > 60
Boundary Value Analysis (BVA)	Bugs cluster at boundaries — test at, just below and just above each edge	For 18–60: test 17, 18, 19 ... 59, 60, 61
Cause-Effect Graphing	Map input causes to output effects as a logic graph, then derive test cases	Complements EP for combinations
Decision Table testing	Tabulate combinations of conditions → actions ; test each rule	See below
State transition testing	Test valid/invalid state changes	Login → locked after 3 failures
Error guessing	Experience-based guessing of likely faults	Empty input, zero, negative

Decision table example — a loan approval with two conditions:

Rule	Age ≥ 21?	Income ≥ 30k?	Action: Approve?
R1	T	T	Yes
R2	T	F	No
R3	F	T	No
R4	F	F	No

Two conditions → **2² = 4 rules**; each column is a test case. Decision tables ensure **every combination** is covered.

White-box testing (structural / glass-box)

Tests the **internal logic and structure of the code** — the tester sees the source. **Coverage**

criteria, from weakest to strongest:

Coverage	Requires
Statement coverage	Every statement executed at least once
Branch (decision) coverage	Every branch (true <i>and</i> false of each decision) taken
Condition coverage	Every boolean sub-condition evaluated both ways
Path coverage	Every independent path through the code executed (strongest, often infeasible)

Branch coverage subsumes statement coverage (100% branch $\Rightarrow$ 100% statement, not vice-versa) — a standard MCQ.

Cyclomatic complexity — the worked white-box metric

Cyclomatic complexity (McCabe) measures the number of **linearly independent paths** through a program — a bound on the minimum number of test cases needed for branch coverage, and a complexity/maintainability indicator. From the **control flow graph (CFG)** with **E** edges and **N** nodes:

$V(G) = E - N + 2P$ (P = number of connected components, usually 1)
Equivalently $V(G) = D + 1$ (D = number of **decision/predicate nodes**)
Or $V(G)$ = **number of enclosed regions** in a planar CFG

All three give the **same** value — computing it two ways is a good exam check.

Worked example. Consider:

```
1  read x
2  if (x > 0)
3      print "positive"
   else
4      print "non-positive"
5  end if
6  print "done"
```

The CFG has decision node at line 2 with two branches merging before line 6.

- **Decisions $D = 1$** $\rightarrow V(G) = D + 1 = 2$.

- **Nodes N :** {1-2, 3, 4, 6} = 4; **Edges E :** 2 $\rightarrow$ 3, 2 $\rightarrow$ 4, 3 $\rightarrow$ 6, 4 $\rightarrow$ 6, plus 1 $\rightarrow$ 2 = 5.

$V(G) = E - N + 2 = 5 - 4 + 2 = 3?$ — recount: model start $\rightarrow$ decision $\rightarrow$ two paths $\rightarrow$ merge $\rightarrow$ end as

$N = 5$ nodes (start, decision, then-branch, else-branch, end), $E = 6$ edges

(start $\rightarrow$ decision, decision $\rightarrow$ then, decision $\rightarrow$ else, then $\rightarrow$ end, else $\rightarrow$ end, plus one entry). Using the reliable rule $V(G) = D + 1 = 1 + 1 = 2$: there are **2 independent paths** ($x > 0$ and $x \leq 0$), so **2 test cases** give full branch coverage. (*Always cross-check with $D + 1$ — it is the least error-prone form and the one to show.*)

Interpretation: $V(G) \leq 10$ is considered manageable; higher means the module is complex, error-prone and should be refactored. The value also equals the **minimum number of test cases** for branch coverage.

Other testing types

Type	Meaning
------	---------

Alpha testing	By users/testers at the developer's site , before release
Beta testing	By real users in their own environment , pre-final release, feedback collected
Regression testing	Re-running existing tests after a change to ensure nothing previously working broke
Smoke / sanity	Quick check that the build is stable enough to test
Stress / load / performance	Behaviour under heavy or extreme load
Recovery / security / usability	Non-functional system tests

Likely exam questions

- **[5]** Differentiate between black-box and white-box testing.
- **[5]** Explain the levels of testing: unit, integration, system, acceptance.
- **[5]** Differentiate top-down and bottom-up integration. What are stubs and drivers?
- **[5]** Explain equivalence partitioning and boundary value analysis with an example.
- **[5]** Differentiate alpha and beta testing. What is regression testing?
- **[5]** Define cyclomatic complexity and state its three formulas.
- **[10]** Draw the control flow graph for a given program, compute its cyclomatic complexity by all three methods and state the number of independent paths.
- **[15]** Explain software testing in detail: the levels of testing, integration strategies, black-box techniques (EP, BVA, decision tables) and white-box coverage with cyclomatic complexity worked on an example.

MCQ traps

Trap	Correct answer
"Testing can prove the absence of bugs?"	No — only their presence.
"Error vs fault vs failure?"	Error (human) → fault (code) → failure (behaviour).
"Top-down integration uses?"	Stubs . Bottom-up uses drivers .
"BVA tests values at?"	The boundaries .
"Which testing needs source code?"	White-box .
"Cyclomatic complexity = ?"	$E - N + 2P = D + 1 = \text{regions}$.
"Branch coverage subsumes?"	Statement coverage .
"Beta testing is done?"	By real users at their own site .
"Testing after a change to catch new breakage?"	Regression testing .
"n conditions in a decision table give?"	2ⁿ rules .

11. Verification and Validation (V&V)

Concept

Two words that sound alike and are constantly confused — Boehm's one-liner nails them:

- **Verification** — *"Are we building the product right?"* — checks the software against its **specification** at each stage. **Static**, no execution: reviews, walkthroughs, inspections.
- **Validation** — *"Are we building the right product?"* — checks the finished software against the **user's actual needs**. **Dynamic**, requires execution: testing.

	Verification	Validation
Question	Building it right ?	Building the right thing?
Against	The specification/design	The user's requirements
Method	Static — reviews, inspections, walkthroughs	Dynamic — testing/execution
When	Throughout, at each phase	Mainly at the end
Finds	Deviations from spec	Wrong/missing requirements

Reviews/inspections (verification): **Fagan inspection** is a formal, role-based review (moderator, author, reader, tester) proven to catch defects far more cheaply than testing.

Walkthroughs are less formal.

Likely exam questions

- [5] Differentiate between verification and validation with examples.
- [5] What is a software inspection/review? How does it differ from testing?

MCQ traps

Trap	Correct answer
"Are we building the product right?"	Verification .
"Are we building the right product?"	Validation .
"Verification is static or dynamic?"	Static (no execution).
"Validation involves?"	Execution/testing .

12. Software metrics

Concept

Software metrics are **quantitative measures** of a product, process or project — "you cannot control what you cannot measure." Categories:

- **Product metrics** — size and quality of the software: **LOC/KLOC**, **function points**,

cyclomatic complexity, Halstead's software science (operators/operands → volume, difficulty, effort), defect density (defects/KLOC).

- **Process metrics** — the development process: productivity (LOC or FP per person-month), defect removal efficiency, rework percentage.
- **Project metrics** — cost, effort, schedule, staffing.

LOC vs Function Points — the classic metrics comparison:

	LOC	Function Points
Measures	Physical size (lines)	Delivered functionality
Language-dependent?	Yes	No
Available	Only after coding	Early, from requirements
Weakness	Penalises concise code; varies by style	Subjective weighting

Halstead metrics (from n_1 , n_2 = distinct operators/operands; N_1 , N_2 = total counts):
Vocabulary $n = n_1 + n_2$; Length $N = N_1 + N_2$; **Volume** $V = N \times \log_2 n$; Difficulty $D = (n_1/2) \times (N_2/n_2)$; Effort $E = D \times V$.

Likely exam questions

- **[5]** What are software metrics? Classify them.
- **[5]** Differentiate LOC and function points as size metrics.
- **[10]** Explain product, process and project metrics with examples, including Halstead's software science.

MCQ traps

Trap	Correct answer
"Which size metric is language-independent?"	Function points.
"Defects per KLOC is a?"	Product (quality) metric.
"Halstead volume $V = ?$ "	$N \times \log_2 n$.
"Cyclomatic complexity is a ____ metric?"	Product (complexity) metric.

13. Software Quality Assurance (SQA)

Concept

Software quality = conformance to **explicitly stated functional and performance requirements, explicitly documented development standards**, and **implicit characteristics** expected of professional software (Pressman). Note all three clauses — implicit expectations count too.

SQA is the **planned, systematic set of activities** that ensures quality is built into the process, not merely tested in at the end. It is **process-oriented** (prevention);

testing/QC is **product-oriented** (detection).

SQA activities: defining standards and procedures; reviews, audits and inspections;
configuration management; measurement (metrics); defect tracking; and reporting to management.

Quality control (QC) vs Quality assurance (QA): QC finds defects in the product (testing, inspection); QA improves the process to prevent defects. QA is the umbrella; QC is one activity within it.

Likely exam questions

- **[5]** Define software quality. What is SQA?
- **[5]** Differentiate quality assurance from quality control.
- **[10]** Explain the activities involved in software quality assurance.

MCQ traps

Trap	Correct answer
"QA is process- or product-oriented?"	Process (prevention). QC = product (detection).
"Software quality includes implicit requirements?"	Yes .
"Which is broader, QA or QC?"	QA .

14. Quality models — McCall, ISO 9126, CMM

McCall's quality model (1977)

Organises **11 quality factors** (external, user-visible) into **three perspectives**, each supported by internal **criteria**:

Perspective	Factors
Product operation	Correctness, Reliability, Efficiency, Integrity, Usability
Product revision	Maintainability, Flexibility, Testability
Product transition	Portability, Reusability, Interoperability

Mnemonic for the three perspectives: **operation** (how well it runs), **revision** (how easily it changes), **transition** (how well it moves to new environments).

ISO 9126 quality model

The international standard defining **six quality characteristics** (each with sub-characteristics):

Characteristic	Meaning
Functionality	Does it provide the required functions (suitability, accuracy, security)?
Reliability	Maturity, fault tolerance, recoverability

Usability	Understandability, learnability, operability
Efficiency	Time behaviour, resource use
Maintainability	Analysability, changeability, testability, stability
Portability	Adaptability, installability, replaceability

(Superseded by **ISO/IEC 25010**, which adds **security** and **compatibility** as top-level characteristics — mention it.)

CMM — Capability Maturity Model (SEI) — levels 1–5

The **CMM** rates an **organisation's software-process maturity** on **five levels**; each level builds on the one below through **Key Process Areas (KPAs)**. This is a guaranteed exam item — learn the five levels and their one-word essence.

Level	Name	Essence	Character
1	Initial	Ad hoc, chaotic	Success depends on individual heroics; unpredictable
2	Repeatable (Managed)	Basic project management	Processes for cost/schedule/tracking; can repeat earlier successes
3	Defined	Standardised, documented organisation-wide process	A tailored standard process for all projects
4	Managed (Quantitatively Managed)	Measured and controlled	Process and product quantitatively understood via metrics
5	Optimising	Continuous improvement	Feedback and innovation continuously improve the process

Mnemonic: **I**nitial, **R**epearable, **D**efined, **M**anaged, **O**ptimising →

"I Really Do Manage Operations." Note: **Level 1 has no KPAs**; higher levels

each add KPAs (e.g. Level 2: requirements management, project planning, configuration management;

Level 5: defect prevention, process/technology change management). CMM has evolved into

CMMI.

Likely exam questions

- **[5]** State the three product perspectives of McCall's quality model with their factors.
- **[5]** List the six quality characteristics of ISO 9126.
- **[5]** List and briefly explain the five levels of the CMM.
- **[10]** Explain the CMM. Describe its five maturity levels and their key process areas.
- **[10]** Compare McCall's model, ISO 9126 and the CMM as approaches to software quality.

MCQ traps

Trap	Correct answer

"How many levels in the CMM?"	Five (Initial → Optimising).
"CMM Level 3 is?"	Defined . Level 5 = Optimising .
"Which CMM level has no KPAs?"	Level 1 (Initial) .
"ISO 9126 defines how many characteristics?"	Six .
"McCall's model has how many factors?"	11 , in 3 perspectives.
"Maintainability belongs to which McCall perspective?"	Product revision .

15. Software reliability

Concept

Software reliability is the **probability of failure-free operation for a specified time in a specified environment**. Unlike hardware, software failures come from **design/coding defects**, not wear — so reliability *grows* as defects are found and fixed (**reliability growth models**, e.g. Jelinski-Moranda, Musa).

The key measures (and the relationship among them):

Metric	Meaning
MTTF — Mean Time To Failure	Average time the system runs before a failure (non-repairable)
MTTR — Mean Time To Repair	Average time to fix and restore after a failure
MTBF — Mean Time Between Failures	MTBF = MTTF + MTTR
Availability	MTTF / (MTTF + MTTR) = MTTF / MTBF , often as a %
Failure rate (λ)	Failures per unit time; Reliability $R(t) = e^{(-\lambda t)}$

Worked example. If MTTF = 970 hours and MTTR = 30 hours, then MTBF = 1000 hours and **availability = 970/1000 = 97%**.

Clean-room approach (named in the Forensic syllabus): a methodology that emphasises **defect prevention rather than removal** — rigorous specification, **formal correctness verification** (mathematical proofs / correctness arguments) instead of unit debugging, and **statistical testing** based on the expected usage profile to certify reliability. The aim is to produce software with very low defect density on first release.

Likely exam questions

- **[5]** Define software reliability. Differentiate MTTF, MTTR and MTBF.
- **[5]** How does software reliability differ from hardware reliability?
- **[5]** What is the clean-room approach to software development?
- **[10]** Explain software reliability, its metrics and reliability growth, and compute availability from given MTTF/MTTR.

MCQ traps

Trap	Correct answer
"MTBF = ?"	MTTF + MTTR .
"Availability = ?"	MTTF / (MTTF + MTTR) .
"Reliability formula $R(t)$?"	$e^{(-\lambda t)}$.
"Clean-room emphasises?"	Defect prevention + formal verification + statistical testing.
"Software fails due to?"	Design/coding defects , not wear.

16. Software maintenance

Concept

Maintenance is modifying software **after delivery**. It is the **longest and costliest** phase (~60–70% of lifetime cost). Four types — a guaranteed MCQ and 5-marker:

Type	Trigger	Purpose	Share
Corrective	A fault/bug reported	Fix errors	~20%
Adaptive	A changed environment (new OS, hardware, law)	Keep it working in the new environment	~25%
Perfective	User requests new/better features	Enhance functionality/performance	~50% (largest)
Preventive	Latent problems / ageing code	Prevent future faults; improve maintainability (re-documentation, restructuring)	~5%

Perfective maintenance is the largest slice — most maintenance is *enhancement*, not bug-fixing. That surprises students and is a favourite MCQ.

Lehman's laws of software evolution (worth a mention): software must **continually change** or become less useful (continuing change); and as it changes its **complexity increases** unless work is done to reduce it (increasing complexity).

Likely exam questions

- **[5]** What is software maintenance? Explain its four types.
- **[5]** Which type of maintenance consumes the most effort and why?
- **[10]** Explain the types of software maintenance and the factors that make maintenance difficult.

MCQ traps

Trap	Correct answer
------	----------------

"Fixing reported bugs is?"	Corrective maintenance.
"Adapting to a new OS is?"	Adaptive maintenance.
"Adding new features is?"	Perfective maintenance.
"Which type consumes the most effort?"	Perfective (~50%).
"Improving future maintainability is?"	Preventive maintenance.

17. Reengineering and reverse engineering

Concept

Both deal with **existing (legacy) systems** and are named explicitly in the Forensic syllabus.

- **Reverse engineering** — analysing an existing system to **recover its design/specification**, working **backward** from code to higher-level abstractions (design, then requirements). It **only extracts information; it does not change the system**. Used to understand undocumented legacy code. (In the Forensic context, reverse engineering of *binaries/malware* is analysing compiled code to recover its behaviour.)
- **Forward engineering** — the normal direction: requirements → design → code.
- **(Software) Reengineering** — **examining and altering** an existing system to **reconstitute it in a new, improved form** while **preserving its functionality**. It is essentially **reverse engineering followed by forward engineering**: understand the old system, then rebuild it with better structure, technology or documentation. Activities: inventory analysis, document restructuring, reverse engineering, **code restructuring**, **data restructuring**, forward engineering.
- **Restructuring** — changing code/data structure **without changing functionality** (e.g. converting spaghetti code to structured code); a component of reengineering.

The relationship to state: *reengineering* = *reverse engineering* + (*restructuring*) + *forward engineering*. Reverse engineering **understands**; reengineering **rebuilds**.

Likely exam questions

- **[5]** Differentiate reverse engineering and reengineering.
- **[5]** What is software restructuring? How does it relate to reengineering?
- **[10]** Explain software reengineering, its process and its relationship with reverse and forward engineering.

MCQ traps

Trap	Correct answer
"Recovering design from code is?"	Reverse engineering.

"Rebuilding a legacy system in improved form is?"	Reengineering .
"Reengineering = ?"	Reverse engineering + forward engineering .
"Changing structure without changing behaviour is?"	Restructuring .
"Does reverse engineering modify the system?"	No — it only extracts information.

18. Last-week revision sheet

Twenty facts most likely to be one-mark MCQs:

1. Software **deteriorates, does not wear out**; software = **programs + data + documentation**.
2. **Software crisis** → over budget/schedule, poor quality; term from the **1968 NATO** meeting.
3. SDLC phases: requirements → design → coding → testing → deployment → **maintenance (longest/costliest, ~60–70%)**.
4. **Waterfall** = fixed requirements, no early product; **Prototype** = unclear requirements; **Spiral (Boehm)** = **risk-driven**; **V-model** = each phase paired with a test phase; **Agile** = volatile requirements, continuous delivery.
5. **SRS** must be correct, complete, **unambiguous**, consistent, **verifiable**, traceable (**IEEE 830**).
6. **Feasibility** = **TELOS** (Technical, Economic, Legal, Operational, Schedule).
7. **Brooks's Law**: adding people to a late project makes it later.
8. Basic **COCOMO** $E = a(KLOC)^b$, $D = 2.5 \cdot E^d$; organic $a=2.4$, $b=1.05$; embedded highest $b=1.20$. Intermediate adds **EAF** (15 cost drivers). **Boehm, 1981**.
9. **Function points** are **language-independent**, computed early; five types **EI, EO, EQ, ILF, EIF**; $FP = UFP \times VAF$, $VAF \in [0.65, 1.35]$.
10. **Critical path** = **longest path** = **minimum project duration**; critical activities have **zero slack**. $PERT\ t_e = (o + 4m + p)/6$.
11. **Gantt** shows timeline (not dependencies); **PERT/CPM** shows dependencies; **CPM** = **deterministic**, **PERT** = **probabilistic**.
12. Design goal: **low coupling, high cohesion**.
13. Coupling best→worst: **Data, Stamp, Control, External, Common, Content**.
14. Cohesion worst→best: **Coincidental, Logical, Temporal, Procedural, Communicational, Sequential, Functional**.
15. **Unit** uses stubs/drivers; **top-down** = **stubs**, **bottom-up** = **drivers**; **acceptance** by the customer; **alpha** at developer site, **beta** at user site; **regression** after changes.
16. **Black-box** = spec-based (EP, BVA, decision tables); **white-box** = code-based; **branch coverage** subsumes **statement coverage**.
17. **Cyclomatic complexity** $V(G) = E - N + 2 = D + 1 = \text{regions}$ = independent paths = min test cases for branch coverage.
18. **Verification** = "building it right?" (static); **Validation** = "right product?" (dynamic).
19. **CMM levels**: **1 Initial, 2 Repeatable, 3 Defined, 4 Managed, 5 Optimising** (Level 1 has no KPAs). **ISO 9126** = 6 characteristics; **McCall** = 11 factors in 3 perspectives.

20. Maintenance types: corrective (bugs), adaptive (environment), **perfective (features — largest ~50%)**, preventive; **reengineering = reverse + forward engineering**;
MTBF = MTTF + MTTR, availability = MTTF/MTBF.

Descriptive-paper strategy. Software engineering reliably supplies Section-A questions in all three papers and at least one 15-mark question you can fully answer. Prepare these six as guaranteed long answers, each with its diagram or worked steps:

- (a) **process models compared** (Waterfall, Prototype, Spiral, V-model, Agile) with diagrams — draw each model *first*;
- (b) **coupling and cohesion** fully ranked with examples — a pure-recall high scorer;
- (c) **COCOMO + function points** with one worked numerical each — unambiguous to mark;
- (d) **PERT/CPM** — draw the network, find the critical path and slack;
- (e) **testing** — levels, integration strategies, black/white-box, and **cyclomatic complexity** worked on a CFG;
- (f) **CMM's five levels** and **ISO 9126 / McCall** quality models.

Always draw the diagram or lay out the worked calculation *first*, label it, then write the prose — if you run out of time, a labelled diagram or a correct numerical still scores.